between you and your neighbors in the room, how many different languages do you speak?

Human not code

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Threading

Updates

1. MP5 Allocator due tonight
2. MP6 Wallet out today (start after Thursday's lecture)
3. Exam 2 sign up open
4. HW 5 out now, due Wednesday

Threading

Today's LGs - Expand your mental model of a program to include multiple concurrent threads.

Be able to explain the difference between a thread and a process

Be able to reason about a multithreaded program to figure out

- Possible outputs
- If there is a race condition

Agenda

1. Big Picture Perspective

2. Processes

3. Threading
   a. Race Conditions

4. Synchronization in Programming

Big Picture Perspective

Limitations of Reality

- resources are limited
- things don't happen instantly
- dependencies

System

- collection of components that interact

Non-Computer System Example

- make tea
- cake eggs in, pour into pan, bake
- 1 baking sheet

A Computer as a System

Memory, disk, CPU ...

limited resources, things take time, thing have dependencies

- virtual memory, processes, context switching, the OS

From Last Time

Process - running instance of a program

- VM
- Page table
- reg values
- PC values

OS -

context switching

sys calls

What prints?

main.c

```c
#include <stdio.h>
#include <unistd.h>

unsigned int calc(int n){
    unsigned int ret = 1;
    for(int i = 4; i > 0; i--){
        ret = i * ret;
    }
    return ret;
}

int main(int argc, char *argv[]) {
    sleep(3); // Pauses for 3 seconds
    int intValue = argv[1][0] - '0';
    printf("%i", calc(intValue));
}
```

4·3·2·1 = 24

argv[1] = "4\0"

0 65
1 66
2 67
3 68
4 69

```
gcc main.c
./a.out 4
```

clicker.cs.illinois.edu

Q2

~Code~
340

main.c

```c
#include <stdio.h>
#include <unistd.h>

unsigned int calc(int n){
    unsigned int ret = 1;
    for(int i = n; i > 0; i--){
        ret = i * ret;
    }
    return ret;
}

int main(int argc, char *argv[]) {
    sleep(3); // Pauses for 3 seconds
    int intValue = argv[1][0] - '0';
    printf("%i", calc(intValue));
}
```

```
gcc main.c
```

```
./a.out 3
```

```
./a.out 4
```

Process Vs. Threads

Process

VM

PC

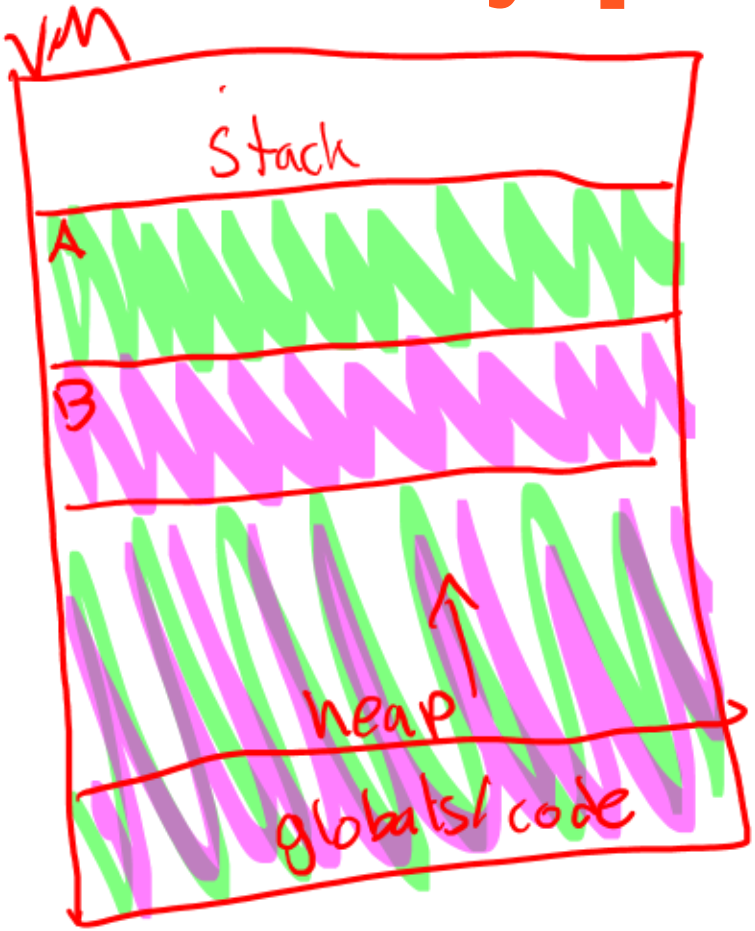
reg values

Thread - every process has atleast 1 thread

- PC
- reg values

X same VM

Virtual Memory Space of a Process with Threads



2 threads
A, B

<u>own</u> stack

<u>Share</u>
code
heap
globals

Thread Example

```
gcc main.c
./a.out
```

```c
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

var

1.

thread address

settings

function name

parameters Void *

OS CPU Scheduling

- Works on a thread level

CPU can run 1 pc, reg, VM

./a.out → Process1 – main thread
– 2nd thread

./a.out → Process2 – main thread
– 2nd thread

P1-M P1-2 P2-M P2-2

VM VM

```
main.c
1  #include <stdio.h>
2  #include <pthread.h>
3
4  void* thread_function(void* arg) {
5      printf("Hello from the new thread!\n");
6      return NULL;
7  }
8
9  int main() {
10     pthread_t thread;
11
12     // Create a new thread
13     if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
14         perror("Failed to create thread");
15         return 1;
16     }
17
18     printf("Hello from the main thread!\n");
19
20     // Wait for the new thread to finish
21     pthread_join(thread, NULL);
22
23     printf("Thread has finished.\n");
24 }
```

pc, reg values.

P1-M → P1-2 easier

P1-M → P2-M harder

What are the max number of threads that could be running concurrently if I run this program from 5 terminals at the same time?

clicker.cs.illinois.edu

Q3

~Code~
340

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

./a.out
./a.out
./a.out
./a.out
./a.out

Max = 10

How many different page tables would the computer need to store to run this program from 5 terminals concurrently?

clicker.cs.illinois.edu

Q4

~Code~
340

5

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

With a 1 core CPU how many processes could I run in parallel?

clicker.cs.illinois.edu

Q5

~Code~
340

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

1

With a 1 core CPU how many threads could I run in parallel?

clicker.cs.illinois.edu

Q6

~Code~
340

main.c

```c
#include <stdio.h>
#include <pthread.h>

void* thread_function(void* arg) {
    printf("Hello from the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread;

    // Create a new thread
    if (pthread_create(&thread, NULL, thread_function, NULL) != 0) {
        perror("Failed to create thread");
        return 1;
    }

    printf("Hello from the main thread!\n");

    // Wait for the new thread to finish
    pthread_join(thread, NULL);

    printf("Thread has finished.\n");
}
```

1

Threading Example

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *compute(void* arg) {
    int* ptr = (int*)arg;
    printf("my value is %i\n", *ptr);
    return NULL;
}

int main() {
    pthread_t threads[2];
    int args[2] = {0, 1};

    for (int i = 0; i < 2; i++) {
        if (pthread_create(&threads[i], NULL, compute, &(args[i])) != 0) {
            perror("Failed to create thread");
            return 1;
        }
    }

    //wait for both threads to finish
    for (int i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }
}
```

my value is 1
my value is 0

address

Thread 1
address of 0 in args

Thread 2
address of 1 in args

What will this code print?

clicker.cs.illinois.edu

Q7

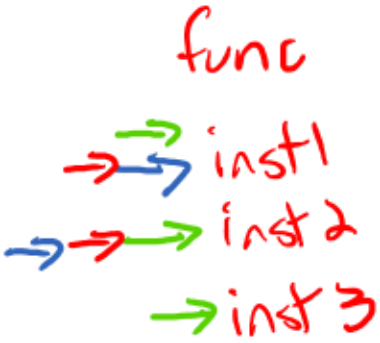
~Code~
340

```
5  void *sort(void* arg) {
6      int *ptr = (int*)arg;
7      sleep(*ptr);
8      printf("%i ", *ptr);
9      return NULL;
10 }
11
12 int main() {
13     int arr[6] = {5, 10, 1, 4, 2, 7};
14     pthread_t threads[6];
15
16     for (int i = 0; i < 6; i++) {
17         if (pthread_create(&threads[i], NULL, sort, &(arr[i])) != 0) {
18             perror("Failed to create thread");
19             return 1;
20         }
21     }
22
23     //wait for both threads to finish
24     for (int i = 0; i < 6; i++) {
25         pthread_join(threads[i], NULL);
26     }
27 }
```

pause for x seconds

Big Idea

The OS can context switch threads between machine instructions.



```
int arr[2] = {0, 0};
int counter = 4;
void* compute(void* arg){
    for(int i = 0; i < 2; i++){
        if(arr[i] == 0){
            arr[i] = counter;
            counter++;
        }
    }
    return NULL;
}

int main() {
    pthread_t threads[2];
    for(int i = 0; i < 2; i++){
        if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
            return 1;
        }
    }
    for(int i = 0; i < 2; i++){
        pthread_join(threads[i],NULL);
    }
    printf("%i, %i", arr[0], arr[1]);
}
```

global

3, 2 ?

arr = [3, 2]

i = 0 1

3, 2

i = 0 1

clicker.cs.illinois.edu

Q8

~Code~
340

Could this code print "3, 2"

yes

Race Condition

Things are racing to access shared resources causing an underterminisic result.

- multiple threads
- shared resource

Is there a Race Condition?

```
5  void* compute(void* arg){
6      int arr[2] = {0, 0};
7      int counter = 1;
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             arr[i] = counter;
11             counter++;
12         }
13     }
14     return NULL;
15 }
16
17 int main() {
18     pthread_t threads[2];
19     for(int i = 0; i < 2; i++){
20         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
21             return 1;
22         }
23     }
24     for(int i = 0; i < 2; i++){
25         pthread_join(threads[i],NULL);
26     }
27     printf("%i, %i", arr[0], arr[1]);
28 }
```

☑ Threads

☒ shared resources

clicker.cs.illinois.edu

Q9

~Code~
340

Is there a Race Condition?

```
5  int arr[2] = {0, 3};
6  void* compute(void* arg){
7      int res = 5;
8      for(int i = 0; i < 2; i++){
9          if(arr[i] == 0){
10             res = res + 3;
11         }
12     }
13     return NULL;
14 }
15
16 int main() {
17     pthread_t threads[2];
18     for(int i = 0; i < 2; i++){
19         if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
20             return 1;
21         }
22     }
23     for(int i = 0; i < 2; i++){
24         pthread_join(threads[i],NULL);
25     }
26 }
```

☑ Threads

☑ shared resource

NO

check for race condition

clicker.cs.illinois.edu

Q10

~Code~ 340

Is there a Race Condition?

```
int arr[2] = {0, 0};
void* compute(void* arg){
    int res = 5;
    for(int i = 0; i < 2; i++){
        if(arr[i] == 0){
            arr[i]++;
        }
    }
    return NULL;
}

int main() {
    compute(NULL);
    compute(NULL);
    compute(NULL);
    compute(NULL);
    printf("%i, %i", arr[0], arr[1]);
}
```

☒ threads

☑ shared resource

NO

clicker.cs.illinois.edu

Q11

~Code~ 340

Race Condition Example

```c
int arr[2] = {0, 0};
int counter = 1;
void* compute(void* arg){
    for(int i = 0; i < 2; i++){
        if(arr[i] == 0){
            arr[i] = counter;
            counter++;
        }
    }
    return NULL;
}

int main() {
    pthread_t threads[2];
    for(int i = 0; i < 2; i++){
        if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
            return 1;
        }
    }
    for(int i = 0; i < 2; i++){
        pthread_join(threads[i],NULL);
    }
    printf("%i, %i", arr[0], arr[1]);
}
```

Synchronization

Idea - make code predictable

threads take turns on critical code

Mutex - mutual exclusion lock

Make mutex

get mutex (lock)
critical code
Release mutex (unlock)

Static Mutex Example

```c
pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
int arr[2] = {0, 0};
int counter = 1;
void* compute(void* arg){
    pthread_mutex_lock(&mut);
    for(int i = 0; i < 2; i++){
        if(arr[i] == 0){
            arr[i] = counter;
            counter++;
        }
    }
    pthread_mutex_unlock(&mut);
    return NULL;
}

int main() {
    pthread_t threads[2];
    for(int i = 0; i < 2; i++){
        if(pthread_create(&threads[i], NULL, compute, NULL) != 0){
            return 1;
        }
    }
    for(int i = 0; i < 2; i++){
        pthread_join(threads[i],NULL);
    }
    printf("%i, %i", arr[0], arr[1]);
}
```

What do mutexes give us?
What do we lose?

lose ~~parall~~ concurrency

avoid race conditions

clicker.cs.illinois.edu

Q12

~Code~
340

Where should I put my mutex lock and unlock to avoid a race condition?

clicker.cs.illinois.edu

Q13

~Code~
340

```
5   int arr[2] = {0, 0};
6   void* compute(void* arg){
7       int res = 5;
8       for(int i = 0; i < 2; i++){
9           if(arr[i] == 0){
10              arr[i]++;
11          }
12      }
13      return NULL;
14  }
15
16  int main() {
17      compute(NULL);
18      compute(NULL);
19      compute(NULL);
20      compute(NULL);
21      printf("%i, %i", arr[0], arr[1]);
22  }
```

lock

unlock

What would happen if I added a mutex?

```
5  void *sort(void* arg) {
6      int *ptr = (int*)arg;
7      sleep(*ptr);
8      printf("%i ", *ptr);
9      return NULL;
10 }
11
12 int main() {
13     int arr[6] = {5, 10, 1, 4, 2, 7};
14     pthread_t threads[6];
15
16     for (int i = 0; i < 6; i++) {
17         if (pthread_create(&threads[i], NULL, sort, &(arr[i])) != 0) {
18             perror("Failed to create thread");
19             return 1;
20         }
21     }
22
23     //wait for both threads to finish
24     for (int i = 0; i < 6; i++) {
25         pthread_join(threads[i], NULL);
26     }
27 }
```

lock

deadlock

unlocked

clicker.cs.illinois.edu

Q14

~Code~
340

wouldn't sort

Big Picture Perspective

Limitations of Reality

System